



UD3. PRÁCTICA 4

ENCRYPTADO MEDIANTE SUBSTITUCIÓN POR CLAVE

OBXECTIVOS

- Coñecer e usar as estruturas de control e de repetición.
- Traballar cos tipos de datos complexos en Java.
- Utilizar clases do sistema.
- Definir e utilizar as miñas primeiras clases.
- Definir e utilizar métodos.
- Compilar e executar aplicacións desde consola e desde o IDE Eclipse.
- Comprobar o fluxo de execución dun programa en Java.
- Traballar cos tipo de dato char e String.
- Utilizar a clase [Character](#).

DURACIÓN ESTIMADA

- 300 minutos.

RECURSOS NECESARIOS

- Computador persoal.
- Java JDK.
- Eclipse IDE.

ENUNCIADO

Nesta práctica imos continuar traballando coa clase String de Java, así que debemos crear unha clase XeradorPorClave que permita encriptar e desencriptar unha mensaxe usando un algoritmo de substitución, concretamente o [cifrado de Vigenère](#), que despraza as letras da mensaxe segundo a clave proporcionada.

Este sistema de cifrado ten as seguintes características .

- ➔ Temos unha mensaxe que so terá 52 letras, 26 en minúsculas e 26 en maiúsculas (abecedario inglés) e espazos.
- ➔ O espazo non se encripta, e as letras maiúsculas e minúsculas trátanse por separado.



- ➔ Temos unha clave que so podería ter letras que usaremos para encriptar a mensaxe.
- ➔ O proceso implica desprazar cada letra da mensaxe segundo o valor da letra correspondente na palabra clave.
- ➔ No caso de que a clave sexa máis pequena que a mensaxe, o algoritmo repite a clave tantas veces como sexa necesario até cubrir a lonxitude da mensaxe.

Exemplo:

Para comprobar o funcionamento do [cifrado de Vigenère](#), usaremos un exemplo práctico:

- **Mensaxe orixinal:** Ola W
- **Palabra clave:** zip

Agora imos ver paso a paso como conseguimos a palabra cifrada:

- A) **Expandir a clave:** A clave debe repetirse para cubrir toda a mensaxe, ignorando os espazos. Se a clave é menor que a mensaxe deberase expandir. Neste caso a mensaxe ten 4 letras (excluíndo o espazo). A clave "zip" repetirase ata ter o mesmo número de caracteres que a mensaxe.
- **Mensaxe orixinal, en minúsculas e sen espazos:** olaw
 - **Palabra clave:** zipz
- B) **Desprazar as letras:** Cada letra da mensaxe desprazarase segundo o valor da letra correspondente da clave. Valores numéricos das letras:
- **Mensaxe:** o (posición 14), l (posición 11), a (posición 0), espazo, w (posición 22)
 - **Clave estendida:** z (posición 25), i (posición 8), p (posición 15), espazo, z (posición 25)

Agora aplicamos o desprazamento para cada letra:

1) Letra da mensaxe: o (14)

- Desprazamento por z (25).
- $o (14) \rightarrow 14 + 25 = 39 \rightarrow$ como superamos o final do alfabeto, facemos $39 \% 26 = 13$ de resto $\rightarrow$ a letra n(13) e como a orixinal está en maiúscula $\rightarrow$ **N (78)**.

2) Letra da mensaxe: l (11)

- Desprazamento por i (8).
- $l (11) \rightarrow 11 + 8 = 19 \rightarrow$ corresponde a **t (19)**.

3) Letra da mensaxe: a (0)

- Desprazamento por p (15).



➤ $a(0) \rightarrow 0 + 15 = 15 \rightarrow$ corresponde a **p (15)**.

4) **Letra da mensaxe: w (22)**

➤ Desprazamento por z (25).

➤ $w(22) \rightarrow 22 + 25 = 47 \rightarrow$ como superamos o final do alfabeto, facemos $47 \% 26 = 21$ de resto $\rightarrow$ a letra v (21) e como a orixinal está en maiúscula $\rightarrow$ **V (78)**.

Axuda:

Para facilitar a realización deste exercicio, podes ter en conta a seguinte información:

➔ **Os caracteres en Java poden representarse como enteiro mediante o seu valor unicode.**

En base a iso temos o seguinte:

◆ **Letras maiúsculas (A-Z):** A (65), B (66), C (67), D (68), E (69), F (70), G (71), H (72), I (73), J (74), K (75), L (76), M (77), N (78), O (79), P (80), Q (81), R (82), S (83), T (84), U (85), V (86), W (87), X (88), Y (89), Z (90).

◆ **Letras minúsculas (a-z):** a (97), b (98), c (99), d (100), e (101), f (102), g (103), h (104), i (105), j (106), k (107), l (108), m (109), n (110), o (111), p (112), q (113), r (114), s (115), t (116), u (117), v (118), w (119), x (120), y (121), z (122).

➔ A clase [Character](#) ten unha serie de métodos para traballar cos caracteres que poden ser de utilidade. Podes investigar no enlace da documentación oficial que se indica.

➔ Para validar a mensaxe, entre outras, podes utilizar unha expresión regular, para iso temos o método **matches()** que ten calquera obxecto da clase String. Por exemplo, unha expresión regular que permita so letras do abecedario inglés, caracteres numéricos ou espazo, podería ser "[0-9a-z]+". Esta expresión, cunha pequena modificación, pode servir para implementar o método **validarMensaxe** que se pide no exercicio.

➔ No contido do material desta unidade 3 tedes un pequeno manual de expresións regulares para probar por liña de comandos en sistemas operativos linux, aínda que a lóxica para definilas é a mesma.

A realizar:

A clase XeradorPorClave debe conter os seguintes métodos:

- **Terá 3 atributos privados:** un para a mensaxe orixinal, outro para a clave e un máis para a mensaxe encriptada. Para todos eles teremos definido o seu método **get** para acceder de xeito seguro.
- **Construtor:** Recibirá dous parámetros para indicar a mensaxe e a clave, e nel realizarase a validación de ambos parámetros, e se cumpren pois entón invocarase aos métodos para encriptar a clase.
- **encriptarMensaxe:** Método privado que permitirá encriptar a mensaxe recibida desprazando



cada letra segundo a posición da correspondente letra da palabra clave.

- **desencriptarMensaxe:** Método público que devolverá a mensaxe ao seu estado orixinal, e deberá facelo a partires da mensaxe encriptada (sen usar da orixinal).
- **validarMensaxe:** Método privado para validar que a mensaxe só contén letras e espazos, devolverá o resultado como un tipo booleano.
- **mostrarMensaxes:** Método público que imprime a mensaxe orixinal, a encriptada e a desencriptada.

Para executar e probar a nosa app tan so teríamos que indicar as seguintes dúas liñas:

```
// Creación dun obxecto de tipo XeradorPorClave
XeradorPorClave xerador = new XeradorPorClave("Ola W", "zip");

// Mostra as mensaxes
xerador.mostrarMensaxes();
```

A saída a iso tería que ser:

```
-- MENSAXES --
Mensaxe orixinal: Ola W
Palabra clave: zip
Mensaxe encriptada: Ntp V
Mensaxe desencriptada: Ola W
```